

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1714

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 50%

Duration : 1 hour

QUESTIONS: 2

Total Marks:40

Annexure A

Integrated Experiments

Code of Conduct:

I **K. Reshma (192565062)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Reshma

INTEGRATED EXPERIMENT SCENARIO

A telecom operator wants an integrated pipeline that first predicts network faults from sensor records such as signal strength, packet loss, call drops and tower load. After a fault-risk prediction is obtained, a learning agent recommends a suitable self-healing action. The complete pipeline is therefore:

Sensor Data → Fault Classifier → Fault Risk → Q-Learning Agent → Self-Healing Action

QUESTION 1 – Decision Tree vs Neural Network Classifier

Implement a Decision Tree classifier and a Neural Network (Multi-Layer Perceptron) classifier on the same network dataset. Compare Accuracy, Precision, Recall and F1-Score. Plot the Decision Tree and the Neural Network architecture (layers/neurons). Justify which model should be deployed for real-time fault monitoring.

1.1 Objective

The objective is to predict whether a telecom network condition is Normal or Faulty using four sensor parameters: signal strength, packet loss, call drops and tower load. Two supervised learning models are trained on the same data and evaluated using the same test set.

1.2 Input Features and Output

Feature	Meaning	Example Range
Signal Strength	Strength of received network signal	-100 to -50 dBm
Packet Loss	Percentage of packets lost	0–30 %
Call Drops	Dropped calls per observation period	0–20
Tower Load	Percentage of tower utilization	0–100 %

1.3 Python Implementation

```
# Telecom Network Fault Detection
# Decision Tree vs Neural Network (MLP)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Sample telecom sensor dataset
np.random.seed(42)
n = 200

signal_strength = np.random.uniform(-100, -50, n)
```

```

packet_loss = np.random.uniform(0, 30, n)
call_drops = np.random.randint(0, 21, n)
tower_load = np.random.uniform(20, 100, n)

# Create fault label using a simple risk rule
fault_score = (
    (signal_strength < -80).astype(int) +
    (packet_loss > 12).astype(int) +
    (call_drops > 8).astype(int) +
    (tower_load > 80).astype(int)
)
fault = (fault_score >= 2).astype(int)

data = pd.DataFrame({
    "SignalStrength": signal_strength,
    "PacketLoss": packet_loss,
    "CallDrops": call_drops,
    "TowerLoad": tower_load,
    "Fault": fault
})

X = data[["SignalStrength", "PacketLoss", "CallDrops", "TowerLoad"]]
y = data["Fault"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# ----- Decision Tree -----
dt = DecisionTreeClassifier(max_depth=4, random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)

# ----- Neural Network / MLP -----
mlp = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(8, 4),
        activation="relu",
        max_iter=1000,
        random_state=42
    ))
])
mlp.fit(X_train, y_train)
mlp_pred = mlp.predict(X_test)

def show_metrics(name, actual, predicted):
    print("\n", name)
    print("Accuracy :", round(accuracy_score(actual, predicted), 3))
    print("Precision:", round(precision_score(actual, predicted, zero_division=0), 3))
    print("Recall   :", round(recall_score(actual, predicted, zero_division=0), 3))
    print("F1-Score :", round(f1_score(actual, predicted, zero_division=0), 3))

show_metrics("Decision Tree", y_test, dt_pred)
show_metrics("Neural Network (MLP)", y_test, mlp_pred)

# Plot Decision Tree
plt.figure(figsize=(16, 8))
plot_tree(
    dt,
    feature_names=X.columns,
    class_names=["Normal", "Fault"],

```

```

        filled=True,
        rounded=True
    )
plt.title("Decision Tree for Telecom Fault Detection")
plt.show()

# Plot Neural Network architecture
layers = ["Input\n4 neurons", "Hidden 1\n8 neurons", "Hidden 2\n4 neurons", "Output\n1 neuron"]
x = np.arange(len(layers))

plt.figure(figsize=(10, 5))
plt.scatter(x, [1]*len(x), s=1500)
for i, label in enumerate(layers):
    plt.text(i, 1, label, ha="center", va="center", fontsize=10)
for i in range(len(layers)-1):
    plt.arrow(i+0.18, 1, 0.64, 0,
              head_width=0.03, head_length=0.05,
              length_includes_head=True)
plt.xlim(-0.6, len(layers)-0.4)
plt.ylim(0.7, 1.3)
plt.axis("off")
plt.title("Neural Network Architecture")
plt.show()

```

1.4 Explanation of the Code

1. A sample telecom dataset is created with 200 observations.
2. The four sensor values are used as input features.
3. A fault label is generated using a simple risk rule. In a real project, this label should come from historical telecom fault records.
4. The dataset is divided into 80% training data and 20% testing data.
5. A Decision Tree with maximum depth 4 is trained.
6. An MLP Neural Network with 4 input neurons, 8 neurons in the first hidden layer, 4 neurons in the second hidden layer and 1 output neuron is trained.
7. Accuracy, Precision, Recall and F1-Score are calculated for both models.
8. The Decision Tree is plotted using `plot_tree()`. A second plot shows the MLP layer structure.

1.5 Performance Comparison

Run the above program to obtain the exact values for the generated dataset. The following table is the format to record the experimental result.

Model	Accuracy	Precision	Recall	F1-Score
Decision Tree	Record from program	Record from program	Record from program	Record from program
Neural Network (MLP)	Record from program	Record from program	Record from program	Record from program

1.6 Model Comparison and Deployment Justification

For real-time telecom fault monitoring, the Decision Tree is recommended when fast prediction, easy interpretation and low computational cost are the main requirements. A Decision Tree can make a prediction by following only a small number of if-else decisions. This is useful in a monitoring system where alerts must be generated quickly.

The Neural Network is useful when the network data becomes large and the relationship between sensor values and faults is complex. It can learn non-linear patterns, but it requires scaling and usually needs more computation and tuning.

Therefore, for a simple real-time monitoring system, I would deploy the Decision Tree because it provides fast inference and clear explanations. If historical data becomes large and complex, the MLP can be evaluated again and selected if it gives significantly better recall/F1-score.

QUESTION 2 – Reinforcement Learning for Self-Healing Actions

Using the fault-risk output from Experiment 1 as the initial state, implement a Q-Learning agent that recommends a self-healing action (for example: Reroute Traffic, Restart Node, Increase Bandwidth) to minimize total network downtime and repair episodes. Show the Q-table after training, the learned policy, and integrate it with the classifier output into a single end-to-end decision pipeline.

2.1 Objective

The Q-Learning agent learns which self-healing action gives the best long-term reward for each network fault-risk state. The three actions used are Reroute Traffic, Restart Node and Increase Bandwidth.

2.2 States and Actions

State	Meaning	Typical Interpretation
0	Low fault risk	Network is nearly normal
1	Medium fault risk	Network needs attention
2	High fault risk	Serious fault condition
3	Critical fault risk	Immediate self-healing required

Actions

Action 0 – Reroute Traffic

Action 1 – Restart Node

Action 2 – Increase Bandwidth

2.3 Reward Design

A simple reward scheme is used. The agent receives a positive reward when the selected action reduces the fault condition and a negative reward when the action is unsuitable. The reward can be improved in a real telecom system using measured downtime, packet loss, service availability and repair time.

2.4 Python Implementation of Q-Learning

```
# Q-Learning for Telecom Self-Healing

import numpy as np
import pandas as pd

np.random.seed(42)

states = 4
actions = 3
```

```

# 0 = Reroute Traffic
# 1 = Restart Node
# 2 = Increase Bandwidth

action_names = [
    "Reroute Traffic",
    "Restart Node",
    "Increase Bandwidth"
]

Q = np.zeros((states, actions))

alpha = 0.1      # learning rate
gamma = 0.9      # discount factor
epsilon = 1.0    # exploration probability
epsilon_min = 0.05
epsilon_decay = 0.995
episodes = 3000

# Reward table: rows = states, columns = actions
# Values can be changed for a real telecom environment.
reward = np.array([
    [2, 1, 2],      # low risk
    [5, 1, 4],      # medium risk
    [7, 6, 3],      # high risk
    [8, 7, 2]       # critical risk
], dtype=float)

def next_state(state, action):
    # Simple simulated environment
    if state == 0:
        return 0
    if state == 1:
        return 0 if action in [0, 2] else 1
    if state == 2:
        return 1 if action in [0, 2] else 0
    if state == 3:
        return 2 if action in [0, 1] else 1

for episode in range(episodes):

    # Start from a random fault-risk state
    state = np.random.randint(0, states)

    for step in range(10):

        # Epsilon-greedy action selection
        if np.random.rand() < epsilon:
            action = np.random.randint(actions)
        else:
            action = np.argmax(Q[state])

        ns = next_state(state, action)
        r = reward[state, action]

        # Q-Learning update
        Q[state, action] = Q[state, action] + alpha * (
            r + gamma * np.max(Q[ns]) - Q[state, action]
        )

        state = ns

```

```

        if state == 0:
            break

        epsilon = max(epsilon_min, epsilon * epsilon_decay)

q_table = pd.DataFrame(
    Q,
    index=["Low", "Medium", "High", "Critical"],
    columns=action_names
)

print("Q-Table after training:")
print(q_table.round(2))

# Learned policy
policy = np.argmax(Q, axis=1)

print("\nLearned Policy:")
for s in range(states):
    print(
        ["Low", "Medium", "High", "Critical"][s],
        "->",
        action_names[policy[s]]
    )

```

2.5 Expected Q-Table and Policy

The exact Q-values depend on the training run and reward design. After training, the highest value in each row indicates the preferred action for that state.

State	Reroute Traffic	Restart Node	Increase Bandwidth
Low	Learned value	Learned value	Learned value
Medium	Learned value	Learned value	Learned value
High	Learned value	Learned value	Learned value
Critical	Learned value	Learned value	Learned value

Policy rule: for each state, choose the action corresponding to the largest Q-value. For example, if $Q[2,0]$ is the largest value in the High-risk row, the learned policy selects Reroute Traffic for a High-risk condition.

2.6 End-to-End Integration: Classifier → Q-Learning

```

# End-to-end decision pipeline

def get_fault_risk(signal_strength, packet_loss, call_drops, tower_load):
    sample = pd.DataFrame([
        "SignalStrength": signal_strength,
        "PacketLoss": packet_loss,
        "CallDrops": call_drops,
        "TowerLoad": tower_load
    ])

    # Classifier output: 0 = Normal, 1 = Fault
    fault_probability = mlp.predict_proba(sample)[0][1]

    # Convert probability into a Q-Learning state
    if fault_probability < 0.25:

```



```

        risk_state = 0          # Low
    elif fault_probability < 0.50:
        risk_state = 1          # Medium
    elif fault_probability < 0.75:
        risk_state = 2          # High
    else:
        risk_state = 3          # Critical

    selected_action = action_names[np.argmax(Q[risk_state])]

    return fault_probability, risk_state, selected_action

# Example sensor reading
probability, state, action = get_fault_risk(
    signal_strength=-88,
    packet_loss=18,
    call_drops=12,
    tower_load=91
)

print("\nEND-TO-END RESULT")
print("Fault Risk Probability :", round(probability, 3))
print("Risk State           :", state)
print("Recommended Action    :", action)

```

2.7 How the Integrated Pipeline Works

Step 1: The telecom tower supplies sensor values such as signal strength, packet loss, call drops and tower load.

Step 2: The trained classifier predicts the probability of a network fault.

Step 3: The probability is converted into a risk state: Low, Medium, High or Critical.

Step 4: The risk state is given to the trained Q-Learning agent.

Step 5: The agent checks the Q-table and selects the action with the highest Q-value.

Step 6: The selected action is sent to the self-healing system.

Step 7: The real result of the action can be used as a reward for further learning.

2.8 Example End-to-End Flow

Example sensor values: Signal Strength = -88 dBm, Packet Loss = 18%, Call Drops = 12, Tower Load = 91%.

These values indicate a high-risk network condition. The classifier produces a fault-risk probability. The probability is converted into a Q-Learning state. The Q-table is then consulted and the action with the maximum learned Q-value is selected.

2.9 Result

Thus, a Decision Tree and Neural Network classifier were implemented for telecom network fault prediction and evaluated using Accuracy, Precision, Recall and F1-Score. A Q-Learning agent was then implemented to select self-healing actions based on the predicted fault risk. The complete system forms an end-to-end intelligent network management pipeline:

Sensor Data → Fault Prediction → Risk State → Q-Learning → Self-Healing Action

2.10 Conclusion

The integrated approach can reduce manual intervention in telecom network management. The classifier detects abnormal network conditions, while reinforcement learning chooses an appropriate corrective action. For a lightweight real-time system, the Decision Tree is a practical choice because it is fast and interpretable. Q-Learning adds adaptive decision making so that the system can learn better self-healing actions from network feedback.